

Чтобы Medium работал, мы регистрируем пользовательские данные. Используя Medium, вы соглашаетесь с нашей Политикой конфиденциальности, в том числе с политикой использования файлов cookie.

★ История только для участников

База данных +

Масштабирование базы данных +

Распределенные системы +

Разработка программного обеспечения +

Масштабируемость +

Открыть в приложении ↗

Регистрация

Вход

Medium

🔍 Search



инженер

6 минут чтения · 13 февраля 2026 года



Арвинд Кумар

Подписаться



Слушай



Поделиться

Современные системы дают сбой не потому, что `SELECT *` были плохо написаны.

Они дают сбой, когда трафик за одну ночь вырастает в 10 раз.

Когда маркетологи запускают кампанию.

Когда один корпоративный клиент заключает сделку.

Когда объем данных растет быстрее, чем кто-либо мог предсказать.

Большинство сбоев в работе баз данных вызваны **проблемами с масштабированием**, а не ошибками в корректности работы.

Вы можете написать идеальную бизнес-логику, следовать принципам чистой архитектуры, но все равно столкнетесь с перебоями в работе, если архитектура базы данных не будет подстраиваться под нагрузку.

,10 важнейших стратегий масштабирования баз данныхВ этом руководстве подробно рассматриваются с точки зрения реальной жизни, архитектурных компромиссов и практических примеров использования. Если вы

разрабатываете системы, которые должны выдерживать реальный
производс).

Чтобы Medium работал, мы регистрируем пользовательские
данные. Используя Medium, вы соглашаетесь с нашей Политикой
конфиденциальности, в том числе с политикой использования
файлов cookie.

Полная вер
микросерв

gu no Java/

To

Чтобы Medium работал, мы регистрируем пользовательские данные. Используя Medium, вы соглашаетесь с нашей Политикой конфиденциальности, в том числе с политикой использования файлов cookie.

Focusing on how databases grow under real production load—architectural techniques to handle scale, traffic spikes, and data growth.

1 Vertical Scaling (Scale Up)



Increase CPU, RAM, or disk on a single database instance.
Simple to implement, but limited by hardware and becomes expensive quickly.

Where you see it: Traditional relational databases, official financial systems.

2 Read Replicas



Offload read traffic by replicating data to multiple read-only nodes.
Improves read throughput but introduces replication lag.

Where you see it: Distributed locks, leader election systems.

3 Database Sharding



Split data across multiple databases based on a shard key (userId, region, tenant).
Enables horizontal scaling.

Where you see it: Distributed shared memory systems.

4 Partitioning



Divide tables into smaller logical chunks (range, list, hash).
Improves query performance and maintenance operations.

Where you see it: DynamoDB, Cassandra, DNS, large-scale caches.

5 Caching Layer



Serve frequently accessed data from in-memory stores like Redis or Memcached.
Reduces database load dramatically but requires cache consistency strategies.

6 CQRS (Read / Write Separation)



Use different models or databases for reads and writes.
Optimizes performance and scalability for read-heavy systems.

7 Denormalization



Store redundant data to reduce expensive joins.
Improves read performance at the cost of write complexity.

6 CQRS (Read / Write Separation)



Use different models or databases for reads and writes.
Optimizes performance and scalability for read-heavy systems.

9 Index Optimization



Carefully design indexes based on query patterns.
Incorrect indexing can slow down writes and increase storage usage.

10 Index Optimization



Carefully design indexes based on query patterns.
Incorrect indexing can slow down writes and increase storage usage.

Why this matters: Most database outages are scaling failures, not correctness bugs. Choosing the wrong scaling strategy early can limit your system's future growth.

Чтобы Medium работал, мы регистрируем пользовательские данные. Используя Medium, вы соглашаетесь с нашей Политикой конфиденциальности, в том числе с политикой использования файлов cookie.

Чтобы прочитать статью полностью, создайте аккаунт.

Автор сделал эту статью доступной только для участников Medium.
Если вы впервые на Medium, создайте новую учетную запись, чтобы прочитать эту статью на нашем сайте.

Продолжить в приложении

Или продолжить в мобильной версии сайта



Зарегистрируйтесь в Google



Зарегистрируйтесь на Facebook



Зарегистрируйтесь по электронной почте

У вас уже есть аккаунт? [Войти](#)



Чтобы Medium работал, мы регистрируем пользовательские данные. Используя Medium, вы соглашаетесь с нашей Политикой конфиденциальности, в том числе с политикой использования файлов cookie.

low



Автор: A

6,1 тыс. подписчиков · 120 подписчиков

Штатный инженер (от 13 лет), помогающий инженерам успешно проходить собеседования по системному проектированию и расти до штатных должностей. Реальные истории, а не теория. Наставничество 1:1 → codefarm.in/mentorship

Чтобы Medium работал, мы регистрируем пользовательские данные. Используя Medium, вы соглашаетесь с нашей Политикой конфиденциальности, в том числе с политикой использования файлов cookie.

Чтобы Medium работал, мы регистрируем пользовательские данные. Используя Medium, вы соглашаетесь с нашей Политикой конфиденциальности, в том числе с политикой использования файлов cookie.